

CS 473 (Spring 2025)

Midterm 1 Practice Problems

Short Questions

In the exam, there will be a few short questions covering different topics, some testing conceptual understanding (so it'd be good to review the lecture materials). Some examples:

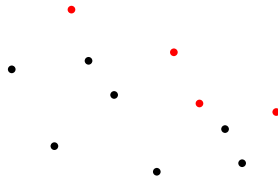
1. Solve the following recurrence (i.e., give tight Θ bound): $T(n) = 8T(n/2) + n^2$ if $n \geq 2023$, and $T(n) = 473$ if $n < 2023$. Justification is not necessary.
2. Solve the following recurrence (i.e., give tight Θ bound): $T(m, n) = 2T(m/2, n) + mn$ if $m \geq 2$, and $T(n) = n$ if $m = 1$. Justification is not necessary.
3. True or False: Strassen's algorithm for matrix multiplication is faster than the trivial algorithm for all input size n and for all input matrices? Briefly justify your answer.
4. Is the dynamic programming algorithm for 0/1 knapsack from class a polynomial-time algorithm? Briefly justify your answer.
5. Give one example of a problem where the Monge property helps speed up dynamic programming.

Divide-and-Conquer

In midterm 1, we may have 1 or 2 divide-and-conquer problems. In these questions, you should usually give pseudocode with brief explanation, and analyze running time using a recurrence.

1. We are given n points $p_1, \dots, p_n$ in 2D. A point p_i is *dominated* by p_j if $p_i.x < p_j.x$ and $p_i.y < p_j.y$. A *maximal point* is a point p_i that is not dominated by any other points. We want to find all maximal points. Describe an $O(n \log n)$ -time divide-and-conquer algorithm for this problem.

(Note: there are several different $O(n \log n)$ -time algorithms...)



2. Describe an efficient divide-and-conquer algorithm that computes the polynomial $P_n(x) = (x - 1)(x - 2) \cdots (x - n)$. (You may use FFT as a subroutine, and you may assume that arithmetic operations on numbers take constant time each.)
3. Given an $n \times n$ matrix A and a positive integer k , describe an efficient algorithm to compute A^k , i.e., the k -th power of A . You may use Strassen's matrix multiplication algorithm as a subroutine. Analyze the running time of your algorithm as a function of n and k . Your algorithm should work for all k , not necessarily even.

4. For two sets A and B , their sumset $A + B$ is defined as $\{a + b : a \in A, b \in B\}$.

Given a subset S of $\{1, \dots, N\}$ and a number k , we want to compute the sumset $S + S + \dots + S$ with k terms. In other words, we want to generate all numbers that can be expressed as a sum of k (not necessarily distinct) numbers from S . Describe an $O(kN \log(kN))$ -time algorithm. (You may use FFT as a subroutine.)

Dynamic Programming

In midterm 1, we may have 1 or 2 DP problems. In these questions, a complete solution includes: (i) first define your subproblems precisely, (ii) then derive the recursive formula (including base cases) with brief justifications, (iii) determine evaluation order, (iv) write pseudocode to output the optimal value, (v) write pseudocode to output the optimal solution, and (vi) analyze the running time (and sometimes also space). Read the problem statement in the exam carefully, as we may allow you to omit certain parts, e.g., (iv) and (v). We may or may not give suggestions for (i). (And if we give you the formula for (ii) and ask you to do (iii)–(vi), then the question should be straightforward.)

1. Given three strings $a_1 \dots a_n$, $b_1 \dots b_n$, and $c_1 \dots c_n$, describe an efficient dynamic programming algorithm to find the longest common subsequence of the three strings.
2. We are given a sequence of n points $p_1, \dots, p_n$ in 2D, where each point is colored red or blue. We are also given a number k . We want to find a polygonal chain $\langle p_{i_0}, p_{i_1}, \dots, p_{i_\ell} \rangle$ with $1 = i_0 < i_1 < i_2 < \dots < i_{\ell-1} < i_\ell = n$, to minimize the total cost $d(p_{i_0}, p_{i_1}) + d(p_{i_1}, p_{i_2}) + \dots + d(p_{i_{\ell-1}}, p_{i_\ell})$, subject to the constraint that the number of *red* vertices on the chain is *at most* k (there is no constraint on the number of blue vertices used or the total number ℓ of vertices used).

Present an efficient dynamic programming algorithm to solve this problem.

3. Let $\Sigma = \{0, 1\}$. We are given a context-free grammar $G = (V, \Sigma, P, S)$ in Chomsky normal form, i.e., all production rules in P are of the form $A \rightarrow BC$ or $A \rightarrow a$ for some variables $A, B, C \in V$ and terminal $a \in \Sigma$. (There are no productions of the form $A \rightarrow \epsilon$.) We are also given an integer n . We want to decide whether there exists a string of length exactly n that is generated by G .

Present an efficient dynamic programming algorithm to solve this problem. Analyze its running time as a function of n and $|V|$ and $|P|$.

4. Suppose you are given a sequence of non-negative integers separated by $+$ and $\times$ signs; for example:

$$2 \times 3 + 0 \times 6 \times 1 + 4 \times 2$$

You can change the value of this expression by adding parentheses in different places. For example:

$$\begin{aligned} 2 \times (3 + (0 \times (6 \times (1 + (4 \times 2)))))) &= 6 \\ (((((2 \times 3) + 0) \times 6) \times 1) + 4) \times 2 &= 80 \\ ((2 \times 3) + (0 \times 6)) \times (1 + (4 \times 2)) &= 108 \\ (((2 \times 3) + 0) \times 6) \times ((1 + 4) \times 2) &= 360 \end{aligned}$$

Describe a dynamic programming algorithm to compute, given a list of integers separated by $+$ and $\times$ signs, the smallest possible value we can obtain by inserting parentheses.

Your input is an array $A[0..2n]$ where each $A[i]$ is an integer if i is even and $+$ or $\times$ if i is odd. Assume any arithmetic operation in your algorithm takes $O(1)$ time.